1 It's a Date

1.1 Parsing

For each exercise, I first focused on parsing the expected input string into a request. Ex.2 was simple, I just check for the string, then place the associated request (**DayToday**) in the output. Ex.3 is similar.

So far, I had used the alternative operator <|> between the parser for each request. However, I noticed a lot of repetition since each parser was of the form "string x >> pure req". So I refactored to fold a function over a pair of string and request, so I didn't need to write the same parser repeatedly.

```
parseRequest :: Parser Request
parseRequest = foldr (\(str, req) prev -> (string str >> pure req) <|> prev)
  (string "Hello" >> pure Hello)
  [
    ("What day is it?", DayToday),
    ("What day is it tomorrow?", DayTomorrow)
  ]
```

Figure 1: parseRequest using foldr

This didn't last long as I quickly realised this wouldn't work for requests which need to store data (such as a date for Ex.4). So I returned to typing each parser out, though I later switched to using choice rather than a bunch of <|>.

My constructor for Ex.4, **TimeSince Day**, makes use of **Day** from **Data.Time**, but I needed a way to convert a string into it. Getting **int**s for the year, month and day was simple but I could not find any constructors for **Day** that took integers! After a bit of searching I discovered that **Year**, **MonthOfYear** and **DayOfMonth** were type synonyms for **int**, so **fromGregorian** would work.

1.2 Responding

Responding to the time-related requests was simply a case of using **getCurrentTime** from **Data.Time** to find the current time on the user's device, and then converting it appropriately. It was fairly easy to find on Hackage **utctDay** to convert the time to a **Day**, then **dayOfWeek** to convert the **Day** to Monday or Tuesday etc. For Ex.4, I just made use of the **diffDays** function.

2 Self Expression

2.1 Parsing

To parse an expression, I used similar ideas to the parsing lab, making a custom **Expr** type to unambiguously recursively define it. However, here the operator is infix. I know there will always be a number first - after that the expression is (an operator then a number) some number of times >= 0. We ensure calculation left to right by applying the operator first, then checking for another operator.

2.2 Responding

Evaluating an expression is a case of recursively evaluating sub expressions, then applying the relevant operator to the constructor.

```
evalExpr :: Expr -> Int
evalExpr (Plus a b) = evalExpr a + evalExpr b
evalExpr (Minus a b) = evalExpr a - evalExpr b
evalExpr (Times a b) = evalExpr a * evalExpr b
evalExpr (Number x) = x
```

Figure 2: evalExpr

3 Remember, Remember

For this section I worked on the response, then the parser.

3.1 Responding

I chose to use the technique of combining the State and IO monads together to achieve the desired "memory" for Ex.7 and Ex.8. This is done using monad transformers. I knew that I wanted to keep the memory separated from the parsing - the request generated by the parsers should just give the two relevant strings to the logic. Then my response code would need access to a state monad to "read/write" to the stateful value.

In order for the stateful value to be kept through each REPL loop, this meant that I needed **runREPL** to also have access to a state monad, so the value can be threaded into the **respondTo** function. Unfortunately, **runREPL** is not called from within src so I must keep the output as IO(). Thus, I have created a new function **runREPL'** with the output I need, and then **runREPL** just calls **evalStateT** to get the IO() out.

I'm using the **mtl** library to define the output as a type **m** that is constrained to the type classes **MonadState Var m** and **Monad IO m**, so the monad has the functionality of both IO and State. This means I need to use **liftIO** to use the IO functions, but state functions can just be called normally. The **Var** type is a custom type I created in order to store multiple values in one stateful variable. I have a single constructor which uses record syntax to name its arguments, which mimic a variable in imperitive programming.

In the **respondTo** function, I can just use **modify**, **get** and record syntax to "read" and "write" to memory. The arguments to Var are a **Maybe Int** to store the result of the last expression for "that", and a Map (from Data.Map) of strings to strings. To remember a name string is a thing, I just insert that key value pair into the map.

I had to rewrite my expression evaluation to account for it accessing memory and the possibility of failure if "that" is present without a value. So the return type could no longer be just **Int**. **Mtl** doesn't have a definition for Maybe so I don't use transformers for implementing that. For the rewrite, I made use of defining a function **op** in a where, to make the code more readable while still evaluating sub expressions first.

3.2 Parsing

In order to parse for Ex.7, I needed to find the substring " is " and then store characters before and after it as two strings (name and thing). My initial approach was to try and use the 'some' function of Alternative along with a parser for any character (I couldn't find an 'anyChar' but instead found 'anySingle' which is almost the same). However, I later discovered the 'manyTill' function which does exactly the job needed, in likely a cleaner way. Once I had that working, parsing Ex.8 was as simple as manyTill a fullstop.

4 Beyond the Specification

I implemented a number of quality of life features beyond the specification:

- Added a couple alternate ways to trigger the same request. Namely, "Hi" and "Hey" for Hello and "What day is it today?" for DayToday.
- Made punctuation at the end of the request optional for most requests (?, !, . or nothing). Memory requests still need to be a . so I know when the [thing] or [name] is ended.
- Extended time functionality: I extended the answers to asking the day of the week to also print the day of the month (e.g. Today is Monday the 1st of January). Additionally, I added some conditions to Ex.4 to alternatively reply saying the date is today, yesterday, tomorrow, or in the future. I've updated the tests accordingly.
- Added a ClearMemory request triggered by "Forget about [name]."

5 Online Resources

https://www.youtube.com/watch?v=UPb83JiGiIY